

# LingoBridge

## 部署说明

文档编号: DEP\_LingoBridge\_v1.0

项目名称: LingoBridge 中文学习平台

编 写: 曹磊

版本编号: v1.0.0

编写日期: 2026 年 5 月 18 日

参考标准: GB/T 8567—2006

# 目 录

|                      |    |
|----------------------|----|
| 第 1 章 环境要求.....      | 1  |
| 1.1 硬件环境.....        | 1  |
| 1.2 软件环境.....        | 1  |
| 1.3 网络环境.....        | 2  |
| 第 2 章 部署步骤.....      | 4  |
| 2.1 本地开发部署.....      | 4  |
| 2.2 Docker 部署.....   | 4  |
| 2.3 腾讯云公网部署.....     | 6  |
| 第 3 章 部署验证.....      | 7  |
| 3.1 接口健康检查.....      | 7  |
| 3.2 系统冒烟测试.....      | 7  |
| 第 4 章 日常运维与备份.....   | 10 |
| 4.1 日志管理与监控.....     | 10 |
| 4.2 数据备份与恢复.....     | 10 |
| 4.3 回滚与恢复机制.....     | 11 |
| 第 5 章 故障排查与安全加固..... | 13 |
| 5.1 常见故障排查.....      | 13 |
| 5.2 系统安全加固建议.....    | 14 |
| 5.3 生产环境演进建议.....    | 14 |
| 参考文献.....            | 16 |

# 第 1 章 环境要求

为了保障 LingoBridge 中文学习平台在实际教学与学生课后练习场景下的稳定运行与高效服务，本部署说明文档依据计算机软件文档编制规范<sup>[1]</sup>的指导进行编写。本章将从硬件环境、软件环境以及网络环境三个维度，详细定义该平台在生产环境下的最低要求与推荐配置，确保系统能够为哈萨克斯坦留学生提供流畅的跨国中哈语音流分发、课件加载及音频存储服务。

## 1.1 硬件环境

LingoBridge 平台的前后端交互涉及频繁的多媒体音频传输以及课件上传处理，对宿主机服务器的资源吞吐性能有着明确的要求。为了支撑并发教学演示以及课件的快速转换，系统部署的服务器硬件资源需要具备良好的处理能力与吞吐速率。下表定义了系统所需的具体硬件资源指标。

表 1.1 LingoBridge 系统部署硬件资源配置要求

| 资源类别 | 最低配置要求        | 推荐配置要求        | 备注说明                             |
|------|---------------|---------------|----------------------------------|
| 处理器  | 单核 2.0 GHz 以上 | 双核及以上处理器      | 用于处理多路并发音频流和静态资源的分发              |
| 运行内存 | 2 GB 可用空间     | 4 GB 或更高容量    | 支撑多通道 high-fidelity 语音合成与多媒体数据处理 |
| 磁盘存储 | 10 GB 固态硬盘    | 20 GB 或大容量存储  | 存放系统代码、运行日志与师生音频录音文件             |
| 网络带宽 | 1 Mbps 独立带宽   | 5 Mbps 以上共享带宽 | 保障中哈跨境语音传输与高画质课件下载速度             |

在实际的课程演练与教学实践中，若有高并发班级测试，推荐将服务器配置升级至四核处理器与八吉字节运行内存，同时采用独立的高速固态存储阵列，以大幅降低磁盘输入输出等待延迟，并为学生课后高频录音和语音回放提供更高的并发支持。

## 1.2 软件环境

系统底层的稳定性与高效执行直接依赖于宿主机软件栈的合理搭配与版本规范。LingoBridge 后端基于先进的异步非阻塞事件驱动引擎构建，前端采用高性能的单页架构，整体支持宿主机直接运行或高度隔离的容器化封装。下表详细列出了系统部署所依赖的关键软件环境及其版本要求。

需要特别指出的是，当前 LingoBridge 平台处于第一阶段最小可行性产品测试期，为了最大程度简化系统部署复杂度、排除外部第三方组件的不稳定因素，系统内部数据库引擎采用无外部依赖的文件存储架构，所有用户账户、课件索引及音频映射记录均持久化写入宿主机本地的 JSON 格式数据文件 `backend/data/db.json` 中，无需预先安装配置甲骨文关系型数据库、结构化查询语言数据库或缓存系统。

表 1.2 LingoBridge 系统部署软件依赖版本与用途说明

| 软件依赖名称         | 适用最低版本           | 主要功能与部署用途说明                |
|----------------|------------------|----------------------------|
| 操作系统           | Ubuntu 20.04 LTS | 提供底层安全、稳定的服务器系统运行基础环境      |
| Node.js 运行时    | 20.0.0 或更高版本     | 负责后端 Express 应用执行与前端静态资源编译 |
| PM2 进程管理器      | 5.3.0 或更新版本      | 本地直接部署模式下用于应用后台驻留与自动拉起     |
| Docker 引擎      | 20.10.0 以上版本     | 生产容器化部署模式下承载应用微服务与反向代理     |
| Docker Compose | 1.29.0 或更高版本     | 编排后端服务容器与前端反向代理容器的互联互通     |
| Nginx 反向代理     | 1.20.0 以上版本      | 实现多语言静态资源的本地缓存以及跨域反向代理     |

### 1.3 网络环境

在网络边界配置方面，为了保障客户端与服务端的安全、稳定连接，宿主机操作系统及云服务商安全组必须按照最小化暴露原则配置防火墙访问规则。系统对外开放的物理端口仅限于用于远程运维控制的安全外壳协议端口二十二，以及承载浏览器超文本传输协议流量的端口八十与加密传输协议端口四百四十三。

后端应用服务在容器内部或宿主机本地监听的端口默认为三千零一，前端开发调试及静态资源的本地服务监听端口通常分配在三千或四千一百七十四。这部分服务监听端口应仅限内部网络环回接口访问，严禁直接向公网公开，所有来自外部的请求均需通过部署于外网端口的反向代理引擎进行路由中转与会话分发。系统的物理与逻辑网络拓扑结构如附图 1.1 所示。

对于正式面向师生的公网测试环境，部署人员应预先为目标服务器完成可用域名的解析配置，并向权威数字证书颁发机构申请有效的传输层安全协议数字证书，通过在反向代理中绑定域名与证书实现全链路流量加密，从而杜绝由于明文传输导致的口令泄漏及语音文件篡改等潜在安全隐患。

图 1.1 LingoBridge 系统部署物理与逻辑拓扑结构图

## 第 2 章 部署步骤

系统的具体实施与上线流程需要严格遵循软件系统设计规范<sup>[2]</sup>的方法论指导，确保各组件在目标操作系统中能够正确装载并建立通信链路。本章将详细阐述 LingoBridge 平台的本地开发部署、Docker 容器化部署以及腾讯云公网环境下的具体执行步骤。

### 2.1 本地开发部署

本地开发部署模式主要面向日常开发调试、原型功能验证以及快速迭代场景。在此模式下，系统的前后端组件直接搭载于开发人员的宿主机环境中运行，通过本机的 Node.js 运行时和进程管理器进行控制。

第一步，获取项目源码与环境核对。部署人员通过代码管理工具将项目的主干分支源码完整克隆至本地的工作目录。在开始后续操作前，须通过终端命令确认当前系统的 Node.js 及 npm 包管理器版本符合前述软件环境要求。

第二步，依赖包的拉取与装载。部署人员在项目根目录下打开终端，执行依赖项自动装载指令，即运行 `npm install` 命令。该命令将根据根目录下的配置文件，自动从配置的镜像源中拉取前端开发依赖以及后端服务依赖，并将其统一解压并建立软链接于 `node_modules` 目录中。

第三步，环境配置文件的创建与参数调整。部署人员需要将根目录下的环境变量模板文件 `.env.example` 复制一份并重命名为 `.env` 文件。在此文件中，可以根据本地调试环境的具体情况对配置参数进行个性化微调，包括但不限于后端的服务绑定地址、后端服务侦听端口等。

第四步，前端构建与服务常驻运行。对于开发调试，可以直接运行 `npm run dev` 启动 Vite 提供的热重载开发服务器。对于生产模拟，则首先需要执行 `npm run build` 对 React 前端项目进行静态编译，所有优化压缩后的资源将输出至 `dist` 目录。此后，通过运行命令 `npm run pm2:start`，PM2 进程管理器将根据配置文件 `ecosystem.config.cjs` 的定义，在宿主机后台并发拉起 Express 后端 API 服务，实现应用的常驻守护。

### 2.2 Docker 部署

针对多模块协同、环境强隔离的生产环境，推荐采用 Docker 容器化编排的部署方式。此方式可以实现宿主机环境与应用运行环境的彻底解耦，杜绝由于系统依赖库差异导致的不确定性故障。容器化部署的时序活动流如附图 2.1 所示。

在容器化部署架构中，前端打包静态产物被挂载至高性能的反向代理容器中，而后端 Express 微服务则独立封装于 Node 应用容器内。两个容器通过加入同一个自定义的虚拟网桥实现高效的私有通信，从而将直接的端口暴露限制在反向代理边界。系统的多容器编排关键参数配置如表 2.1 所示。

图 2.1 LingoBridge 系统容器化部署与活动执行时序图

表 2.1 LingoBridge 系统容器编排参数说明

| 服务组件名称              | 容器网络端口映射        | 挂载宿主机持久化卷                              |
|---------------------|-----------------|----------------------------------------|
| lingobridge-backend | 3001:3001       | ./backend/data:/opt/backend/data       |
|                     |                 | ./backend/storage:/opt/backend/storage |
| lingobridge-nginx   | 80:80 , 443:443 | ./nginx/conf:/etc/nginx/conf.d         |
|                     |                 | ./dist:/usr/share/nginx/html           |

执行容器化部署的具体步骤如下：首先，在宿主机执行 `npm run build` 对前端单页应用进行打包，确保生成最新的静态资源。其次，进入项目中的 `docker` 编排配置目录，运行 `docker-compose up -d` 命令。该命令将指示 Docker 引擎自动下载基础镜像、构建后端专属运行时镜像、创建私有网桥，并在后台以守护进程模式拉起后端应用容器与 Nginx 服务容器。最后，通过运行容器状态查看命令，确认所有定义的服务均处于运行状态，并检查持久化卷的挂载目录权限，确保后端容器对宿主机挂载的 `backend/data` 目录具有完整的读写权限，以便本地 JSON 文件数据库能够正确持久化。

## 2.3 腾讯云公网部署

将 LingoBridge 平台部署于腾讯云等公网云服务器上，是面向哈萨克斯坦国际学生提供公网测试的必要步骤。此过程需要将本地开发产物上传至云端虚拟机，并配置安全策略与反向代理。

第一步，云端虚拟机的初始化与端口访问控制。在腾讯云控制台购买并初始化一台搭载 Ubuntu 系统的轻量应用服务器或云服务器，并在腾讯云控制台的安全组规则中，增添允许外部访问的超文本传输协议的八十端口与加密超文本传输协议的四百四十三端口，同时确保用于安全登录的二十二端口受到特定源地址的限制。

第二步，应用产物上传与云端解压。部署人员在本地将通过测试的代码仓库打包为压缩文件，通过安全拷贝协议或腾讯云命令行工具将其上传至云端服务器指定的目录中。上传完成后，登录云端终端执行解压操作，并根据云主机的内网域名或公网域名对解压出的 `.env` 文件进行修改，调整对应的跨域请求基准路径。

第三步，云端反向代理与证书绑定。在云主机上安装并启动 Nginx 服务，或者通过云主机上的 Docker Compose 容器集群直接拉起代理服务。为了保障数据的机密性，需要在 Nginx 的配置文件中指定域名解析，并将从腾讯云数字证书中心下载的 SSL 证书映射至代理配置中，使得所有外部对该域名的 HTTPS 访问都能被安全地解密，并正确转发至本地监听于三千零一端口的 Express 后端 API 服务，最终完成高安全性的公网环境部署。



## 第3章 部署验证

在完成系统部署配置与启动操作后，为了评估系统是否具备上线运行条件并验证其功能和非功能特性，必须依据系统测试文档标准<sup>[3]</sup>进行严格的部署验证与冒烟测试工作。本章将详细描述接口级别的自动健康检查方案，以及模拟师生核心业务闭环的系统冒烟测试流程，确保 LingoBridge 平台的前后端组件、存储路径以及持久化机制在真实运行环境中达到设计预期。

### 3.1 接口健康检查

接口级别的自动健康检查主要通过向部署完成的目标服务器发送特定超文本传输协议请求，并核对返回的状态码与响应体数据，来快速评估系统核心组件的在线状态和本地磁盘的读写状态。该检查可在系统重构、升级或日常维护后，由运维工具或部署脚本自动触发执行。

在实际执行接口健康检查时，首先通过命令行工具 `curl` 向后端的应用健康状态探测接口发送超文本传输协议的头部查询请求，例如指向服务器本地三千零一端口的特定应用健康路由，并验证服务器返回的状态码为二百。若该接口响应超时或返回五百系列错误，则表明后端应用服务未成功拉起或其内部初始化依赖发生阻塞。

其次，需要对反向代理的入口点进行验证，通过向服务器公网域名的根路径发送超文本传输协议请求，核对是否能够快速返回表示正常的二百状态码，从而确保反向代理已正确加载静态产物目录，并且反向代理与后端应用容器的网络通路配置无误。

最后，必须通过操作系统的磁盘目录属性查询命令 `ls -ld`，对后端挂载的持久化数据文件夹 `backend/data` 以及多媒体文件暂存文件夹 `backend/storage` 进行权属与权限核查。确保当前运行 Node.js 进程的系统账户或 Docker 容器内的运行账户对上述两个目录拥有完整的写入与修改权限，防止后续业务交互中出现音频文件无法持久化或数据库写入挂起的严重故障。系统的具体部署验证健康检查对照矩阵如表 3.1 所示。

表 3.1 LingoBridge 部署验证健康检查对照表

| 检查项目类别   | 预期响应状态                         | 检测验证命令与参数                                                |
|----------|--------------------------------|----------------------------------------------------------|
| 后端服务健康接口 | HTTP 200 OK                    | <code>curl -I http://localhost:3001/api/v1/health</code> |
| 前端静态入口页面 | HTTP 200 OK                    | <code>curl -I http://localhost/</code>                   |
| 数据目录写入权限 | 目录权属为 <code>node</code> 且具有写权限 | <code>ls -ld backend/data</code>                         |
| 语音文件存放路径 | 目录权属为 <code>node</code> 且具有写权限 | <code>ls -ld backend/storage</code>                      |

### 3.2 系统冒烟测试

在接口级别的基本在线状态得到确认后，部署人员需要执行一套覆盖教师端与学生端核心交互逻辑的轻量级业务冒烟测试。该测试旨在确保全链路数据交互、文件上传与持久化存储

在实际业务场景下的协同正确性。系统部署验证与冒烟测试的具体业务流程如附图 3.1 所示。

1

图 3.1 LingoBridge 部署验证与冒烟测试业务流程图

冒烟测试的具体业务流验证步骤按如下所述的流程依次展开。

首先，进行多角色账户的身份鉴权验证。测试人员打开客户端浏览器，进入平台的登录入口。分别使用预设的系统管理员账户、教师账户以及学生账户进行登录操作。登录请求发送后，系统需在毫秒级内完成散列值对比与身份令牌的分发，前端页面在接收到有效令牌后，应根据角色权限正确跳转至对应的专属控制台界面，以此证明鉴权中间件与账户比对机制运行良好。

其次，教师端的多媒体教学课件上传流验证。测试人员切换至教师角色账户，进入课件

管理模块，点击课件上传控件并选择一个有效的便携式文档格式课件或演示幻灯片课件进行上传。后端接收到多媒体文件流后，需将其完整写入 backend/storage 目录，并在轻量级 JSON 数据库中追加该课件的唯一编号、文件路径及对应的解析状态。此时，前端页面应展示出该课件解析成功的视图，以此验证多媒体上传网关和文件数据库的高速写入性能。

最后，学生端的语音跟读与高频多媒体流回环验证。测试人员使用学生账户登录，进入上述上传课件的跟读演练界面。通过点击汉语文本触发文字转语音服务，核对扬声器能否流利播放出合成的标准普通话音频。随后，测试人员开启浏览器的媒体录制机制，录制一段本人的发音，并点击上传按钮。学生音频文件被提交至后端后，系统应在 backend/storage 中生成对应的音频文件，并在轻量级数据库中为该学生追加一条录音记录。测试人员在列表中点击播放、下载或删除该段录音，验证相关操作能够被服务器正确执行且磁盘文件被同步更新，从而证明前后台多媒体回环与持久化读写流的绝对完整。

## 第 4 章 日常运维与备份

在软件系统的生命周期中，持续的运维管理与高可靠性的备份策略是保障业务连续性的基石。本章将依据计算机软件文档编制规范<sup>[1]</sup>的相关运维设计要求，详细阐述 LingoBridge 平台的日志监控机制、关键数据备份与恢复策略，以及遭遇故障或升级失败时的回滚机制，旨在为系统管理员提供科学、可复现的标准化运维指导手册。

### 4.1 日志管理与监控

日志数据是进行系统性能审计、安全分析以及故障根因追溯的关键信源。LingoBridge 平台在运行期间会产生应用层级的业务逻辑日志、运行容器的系统日志以及反向代理的访问与错误日志。

在本地开发部署或 PM2 直接部署模式下，后端 Express 服务的所有标准输出和标准错误输出流均会被 PM2 进程管理器自动捕获，并写入宿主机用户主目录下的隐藏文件夹中。系统运维人员可以直接通过执行应用封装的快捷脚本指令 `npm run pm2:logs`，实时查阅前后端服务的控制台输出信息，并可以使用 PM2 自带的日志切割插件，对单个日志文件的大小进行硬性限制，防止出现单个日志文件体积过大从而侵占宿主机有限存储空间的情况。

在容器化部署模式下，后端的运行日志与代理网关的访问日志被重定向至各自容器的内部标准输出通道。运维人员在宿主机上通过运行命令 `docker-compose logs -f backend` 能够实时跟踪后端容器的数据流转，而运行 `docker-compose logs -f nginx` 则能清晰捕获前端静态资源的请求命中与状态分布情况。为了保障生产环境的高可用性，宿主机操作系统中应当预先部署系统级的日志分卷工具。该工具可配置为每日定时触发，自动对累积的容器日志进行滚动切分，将过往历史日志进行高比例的压缩打包并迁移至冷存储介质，同时自动清除超出保存期限的历史冗余，以此确保宿主机磁盘文件系统长期处于健康状态。

### 4.2 数据备份与恢复

由于当前版本的 LingoBridge 平台处于第一阶段最小可行性产品迭代期，为了最大限度减少外部数据库服务的运维负担，平台采用了高度聚合的文件型数据存储方案。系统内的状态数据和多媒体资产分布于特定的持久化文件夹中。

数据资产的第一部分为轻量级文件数据库文件夹 `backend/data`，该文件夹内存放着包含用户信息、课件关系矩阵以及学生练习音频索引的 `db.json` 文件，这是系统的状态核心。数据资产的第二部分为多媒体附件存储文件夹 `backend/storage`，该文件夹内存放着教师上传的原始课件文件以及学生跟读提交的语音记录。

针对此架构，数据备份操作无需执行复杂的数据库管理系统导出流程，而只需通过文件系统的打包压缩命令即可实现。系统管理员应在宿主机的任务调度管理器中增设一项每日清晨定

时执行的备份作业。该作业运行时会自动触发打包脚本，将上述两个目录内的所有内容打包为一个以当前执行日期命名的压缩归档文件，命令类似于 `tar -czf backup_data_(date +%Y%m%d).tar.gz backend/data/ backend/storage/`，并将生成的高效压缩包同步转存至独立的异地云备份空间中。

在灾难发生需要进行数据恢复时，其操作同样极为简便。系统管理员首先需要停止当前运行的所有前端与后端应用服务，防止恢复过程中产生并发读写冲突。其次，将目标备份压缩归档文件下载至项目部署的根目录下，使用文件解包命令进行覆盖式解压，恢复 `backend/data` 与 `backend/storage` 的原始目录结构。最后，重新启动应用服务并核对文件所有者权限，即可在极短时间内完成全系统状态的无损恢复。

## 4.3 回滚与恢复机制

当系统执行新版本发布、补丁升级或配置重构后，若在线上冒烟测试或运行监控中发现无法实时排除的阻塞性缺陷，必须立即启动回滚与灾备机制，使应用服务快速恢复至升级前的最后一个已知稳定状态。系统的部署回滚与故障灾备恢复流程如附图 4.1 所示。

版本回滚的执行流程包含代码逻辑回滚与持久化文件状态还原两个层面，其推进过程必须严格保持时序的一致性。

第一阶段，代码逻辑与静态资源回滚。运维人员在项目部署根目录下打开终端，利用版本控制系统，将当前代码仓库的工作副本强制重置为上一个稳定发布版本的特定标签，例如运行 `git checkout v1.0.0` 命令行。在代码仓库成功重置后，本地 **PM2** 部署模式下需要再次执行 `npm install` 以同步更新依赖库，并运行 `npm run build` 对上一个版本的静态资源进行重新打包编译。最后，通过运行重启指令，在后台重新拉起稳定的应用服务。若是容器化部署模式，则只需修改编排配置文件中的镜像版本号，并运行 `docker-compose up -d --build` 触发容器镜像的就地重构与轻量重启。

第二阶段，持久化文件与文件数据库恢复。由于系统新版本在短暂运行期间可能产生了不兼容的数据结构变更，从而污染了 `backend/data/db.json` 数据文件。在代码回滚完毕后，运维人员必须利用解压命令，将升级前一刻自动备份的历史数据归档文件完整解包，彻底覆盖当前发生污染的 `backend/data` 目录，恢复原本的 JSON 数据契约。

代码逻辑与持久化数据回滚完毕后，必须按照部署验证规范，立即对服务健康接口进行并发探测，并进行基础的账户鉴权与上传冒烟验证，在确认系统逻辑与历史数据完全对齐后，方能正式对外宣告回滚完成。

图 4.1 LingoBridge 部署回滚与故障灾备恢复流程图

# 第 5 章 故障排查与安全加固

软件系统上线运行后，会面临复杂的宿主机物理环境变化、外部网络波动以及恶意攻击威胁。为了最大程度降低故障平均修复时间并增强系统在公网环境下的抗风险能力，本章将依据计算机软件文档编制规范<sup>[1]</sup>的系统安全与健壮性设计要求，梳理 LingoBridge 平台部署期间的常见故障诊断矩阵，提供针对生产主机的安全加固建议，并为后续系统从最小可行性产品平滑演进至大规模高可用架构提出具体的改造设计方案。

## 5.1 常见故障排查

在平台部署或运行期间，若系统出现前端资源无法加载、接口请求报错或业务逻辑中断，运维人员需要根据故障表现症状，利用系统命令对关键瓶颈进行快速定位。下表归纳了系统部署验证中常见的四大典型故障及其对应的潜在诱发原因和推荐的排除解决方法。

表 5.1 LingoBridge 部署常见故障诊断与排除方案对照表

| 故障表现症状             | 潜在诱发原因                   | 推荐诊断与排除  |
|--------------------|--------------------------|----------|
| 服务启动失败且提示端口冲突      | 服务器中其他进程占用了 3001 或 80 端口 | 执行端口占用检查 |
| 用户登录或注册提示数据库写入错误   | 本地数据库文件或数据目录无写权限         | 运行权限授予命令 |
| 浏览器请求时提示 502 错误    | 后端接口服务崩溃或内部代理转发端口配置错误    | 检查后端进程运行 |
| 前端静态资源页面加载报 404 错误 | 前端静态打包产物目录缺失或代理路径指定有误    | 验证前端产物目录 |

第一类是服务端口占用故障。如果系统启动失败并抛出监听端口已被绑定的网络异常，运维人员需要使用终端命令 `lsof -i :3001` 或 `netstat -tuln` 查询当前占用三千零一端口的进程识别号，在确认其不是核心系统依赖后，可通过强制终止命令将其杀掉，以释放端口资源供后端 API 服务正常绑定。

第二类是本地数据库写入故障。当发生新用户无法注册或跟读数据无法持久化，且控制台报错提示磁盘写入受阻时，表明 `backend/data/db.json` 数据文件或其所在目录的读写属性发生了错乱。运维人员需在宿主机运行 `chmod -R 755 backend/data` 指令，并使用所有权变更指令 `chown` 将该文件夹的归属权重新划归给当前 Node 进程的执行用户，从而恢复文件数据库的正常读写。

第三类是反向代理网关五零二故障。如果浏览器访问页面时弹出该错误，意味着反向代理服务器已启动但无法建立与后端应用服务器的连接。此时，需要登录云主机，先查看后端 Express 进程是否处于活跃状态，再核查反向代理配置文件中的代理转发目标端口是否与后端监听端口完全一致。如果处于 Docker 容器网络中，还必须确保后端容器与代理容器加入了相同的虚拟子网，且代理转发目标配置为了正确的后端容器主机名。

第四类是前端静态资源加载四零四故障。如果页面显示空白且控制台充满资源缺失的四零四报错，一般是因为前端打包静态产物目录没有被正确挂载，或者是在前端项目构建时配置

的基准路径与代理配置中的静态根目录发生了漂移。运维人员需仔细对齐 Vite 配置文件中的基准路径参数，并确认编译生成的 dist 目录已完整输出并被反向代理的静态根路径所指向。

## 5.2 系统安全加固建议

系统在公网正式开通后，直接暴露在无边界的互联网威胁之下，必须立即执行安全加固操作，以避免服务器沦为网络木马的攻击目标，保护师生的隐私多媒体数据。

首先，在物理与逻辑层级实施严格的端口访问控制。除了用于远程连接的安全外壳协议端口二十二，以及面向师生浏览器提供服务的超文本传输协议端口八十与四百四十三外，应当利用操作系统的网络防火墙或者云厂商提供的虚拟安全组，彻底屏蔽其他非必要物理端口的公网入站流量。特别需要指出的是，后端 API 监听的三千零一端口应当被严格限制为仅允许本地环回接口访问，杜绝任何外部网络直接探测或建立非代理连接的企图。

其次，强制推行全链路传输层安全协议加密。部署人员应从权威机构申请免费的数字证书，并将其配置文件挂载至反向代理中，在配置规则中明确指定仅启用高安全强度的加密套件，并将所有传统的明文超文本传输协议请求强制重定向为加密超文本传输协议请求，确保包括用户登录口令、跟读语音流在内的所有会话数据在跨国公共网络传输过程中不被非法窃听与篡改。

最后，遵循最小权限原则硬化宿主机文件系统。整个 LingoBridge 应用部署目录 /opt/lingobridge 的所有者，应当被设置为系统底层的非特权普通用户，严禁直接使用系统最高权限的超级用户来运行 Node 进程或拉起 Docker 容器，防止黑客通过应用层级的潜在漏洞提权控制整个宿主机系统。同时，对文件型数据库目录进行严格的访问控制，仅向应用进程账户开放读写权限。

## 5.3 生产环境演进建议

当前的 LingoBridge 平台完全基于轻量级的文件数据库和本地磁盘文件存放机制构建，此种架构能够支撑数十人规模的最小可行性测试，但无法满足未来更大规模用户并发、海量语音数据高频吞吐及服务高可用性的长远诉求。为此，系统应当按照如下方向进行高可用演进改造。

在状态存储层面，应当将本地文件数据库 backend/data/db.json 平滑迁移至成熟的关系型数据库管理系统，如开源的高性能对象关系数据库 PostgreSQL。在后端 Node 架构中，引入先进的对象关系映射器如 Prisma 或 TypeORM，这不仅能够大幅提升复杂业务查询的性能，还可以利用关系型数据库自带的行级锁和事务并发机制，彻底消除高并发写入下可能发生的文件锁死和数据覆盖故障。

在多媒体附件存储层面，应当将本地磁盘路径 backend/storage 升级为云服务商提供的分布式对象存储服务，如腾讯云的对象存储服务或亚马逊的简单存储服务。将教师端上传的大文件课件以及学生端提交的语音录音，通过对象存储服务专有的应用程序接口直接上传至分布式的对象存储桶中，不仅能够实现多节点并发的高速分发，还能利用云厂商的多副



本灾备机制保障多媒体资产的零丢失。

在性能与高可用层面，应当在系统架构中引入 Redis 内存键值数据库，将其用于缓存高频访问的课件索引和用户会话数据，以减轻底层关系数据库的压力。此外，可以部署容器集群编排引擎将前后端服务进行水平多实例扩展，在前端设置负载均衡器进行请求的轮询分发，当某一服务节点发生物理故障时可由引擎自动进行健康检测并进行零感知无缝拉起，实现平台真正意义上的高并发、高可用与弹性伸缩。

## 参考文献

- [1] 中华人民共和国国家质量监督检验检疫总局. GB/T 8567—2006 计算机软件文档编制规范[Z]. 2006.
- [2] IEEE. Ieee 1016-2009 standard for information technology—systems design—software design descriptions[Z]. 2009.
- [3] IEEE. Ieee 829-2008 standard for software and system test documentation[Z]. 2008.